Arun Kumar 09-05-2026

SOFTWARE REQUIREMENTS SPECIFICATION

Hotel Management System

Version: 1.0

Date: May 2026

Language: Java 17+

Platform: CLI (Command-Line Interface)

Build Tool: Maven / Gradle

Test Framework: JUnit 5

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Hotel Management System (HMS). The document is intended for developers, testers, and academic evaluators who need a complete understanding of the system's expected behaviour.

1.2 Project Scope

The Hotel Management System is a Java-based command-line application that allows hotel staff to manage rooms, handle guest bookings, refer unavailable guests to partner hotels, and track commission billing for those partner hotels. The system operates entirely in-memory with no persistent database.

1.3 Definitions and Acronyms

Term / Acronym	Definition
HMS	Hotel Management System — the application described in this document
CLI	Command-Line Interface — the text-based user interface of the system
SRS	Software Requirements Specification
SDD	Software Design Document
Room	A bookable unit in the hotel identified by a unique room number
Booking	A reservation of a room for a specific guest and date range
Referral	A record of a guest being directed to an alternative partner hotel
Commission Bill	An invoice generated for a partner hotel based on referrals sent
DD-MM-YYYY	The date format accepted throughout the system
BK-XXX	Format for auto-generated Booking IDs (e.g. BK-001)
REF-XXX	Format for auto-generated Referral IDs (e.g. REF-001)
BILL-XXX	Format for auto-generated Commission Bill IDs (e.g. BILL-001)
KH001 / AH001 / PH001	Pre-loaded partner hotel IDs in the system

1.4 Overview

The remainder of this document is structured as follows: Section 2 gives an overall description of the system. Section 3 details specific functional requirements. Section 4 covers non-functional requirements. Section 5 lists external interface requirements. Section 6 describes system constraints.

Arun Kumar 09-05-2026

2. Overall Description

2.1 Product Perspective

The HMS is a standalone, self-contained Java application. It does not integrate with any external database, payment gateway, or third-party API. All data lives in ArrayList-based in-memory collections for the lifetime of a single session. The system exposes its functionality exclusively through a text-based CLI with an optional AWT-based cheat-sheet window.

2.2 Product Functions — Summary

Module	Key Functions
Room Management	Add room, delete room, list all rooms, list available rooms
Booking Management	Book room (date-range overlap check), cancel booking, view booking, list all bookings
Recommendation Engine	Display partner hotels with available rooms when a requested room is unavailable
Referral Tracking	Record guest referrals to partner hotels; increment partner customer counter
Commission Billing	Generate/update commission bill per partner hotel; mark bill as paid; view bills
Input Validation	Validate room numbers, room types, dates, guest names, IDs, and prices
CLI / Command Handler	Parse user commands, route to services, display cheat-sheet AWT frame

2.3 User Classes

User Class	Description	Access
Hotel Administrator	Primary user; operates the CLI to manage rooms, bookings, and commissions	Full — all commands
Front-Desk Staff	Handles walk-in guests; books rooms and processes referrals	Booking & referral commands
Accounts Staff	Generates and marks commission bills for partner hotels	Commission commands

2.4 Operating Environment

- Java 17 or later (java.time API used throughout)
- Any OS that supports a Java runtime (Windows, macOS, Linux)
- Terminal / command prompt capable of standard I/O
- AWT display available (optional — for cheat-sheet frame)
- JUnit 5 and Maven/Gradle for running the test suite

2.5 Assumptions and Dependencies

- No persistent storage: all data is lost when the application exits.
- Partner hotel data (KH001, AH001, PH001) is hard-coded in RecommendationService.
- The system clock is trusted for date validation and bill generation dates.
- A single user operates the system at a time (no concurrency requirements).
- Room types are restricted to: Single, Double, Deluxe, Suite.

3. Functional Requirements

3.1 Room Management

FR-R01 — Add Room

Attribute	Details
Command	add-room <roomNumber> <roomType> <pricePerNight>
Example	add-room 101 Deluxe 2500
Inputs	roomNumber (positive integer), roomType (Single/Double/Deluxe/Suite), pricePerNight (positive double)
Preconditions	Room number must not already exist in the system
Behaviour	Validates inputs; creates a Room object with isAvailable=true; adds to the in-memory list
Success Output	Room <number> added successfully.
Exceptions	RoomAlreadyExistsException if duplicate; InvalidInputException for invalid inputs

FR-R02 — Delete Room

Attribute	Details
Command	delete-room <roomNumber>
Behaviour	Locates room by number; removes from in-memory list
Success Output	Room <number> deleted successfully.
Exceptions	RoomNotFoundException if room does not exist

FR-R03 — List All Rooms

Attribute	Details
Command	list-rooms
Behaviour	Prints all rooms with number, type, price, and availability status
Edge Case	Prints "No rooms found in the system." if list is empty

FR-R04 — List Available Rooms

Attribute	Details
Command	available-rooms
Behaviour	Prints only rooms where isAvailable == true
Edge Case	Prints "No rooms are available at the moment." if none available

3.2 Booking Management

FR-B01 — Book Room

Attribute	Details
Command	book <roomNumber> <guestName> <checkInDate> <checkOutDate>
Example	book 101 Arun Kumar 10-05-2026 13-05-2026
Date Format	DD-MM-YYYY
Guest Name	Multi-word names supported (e.g. Arun Kumar Sharma)
Validations	1) Guest name not empty 2) Room exists 3) Dates in DD-MM-YYYY 4) Check-in not in past 5) Check-out after check-in 6) No date overlap with existing CONFIRMED bookings
Booking ID	Auto-generated in format BK-XXX (e.g. BK-001, BK-002)
Default Status	CONFIRMED
Overlap Logic	Overlap exists when: checkIn < existingCheckOut AND checkOut > existingCheckIn
Exceptions	InvalidInputException, RoomNotFoundException, InvalidDateException, RoomNotAvailableException
On Not Available	Triggers alternative hotel recommendation flow

FR-B02 — Cancel Booking

Attribute	Details
Command	cancel <bookingId>
Example	cancel BK-001
Behaviour	Sets booking status to CANCELLED; already-cancelled bookings print a notice but do not throw
Side Effect	Cancelled booking dates become available for new bookings (overlap check skips CANCELLED)
Exceptions	BookingNotFoundException if ID not found

FR-B03 — View Booking

Attribute	Details
Command	view-booking <bookingId>
Behaviour	Prints Booking ID, guest name, room number, check-in, check-out, and status
Exceptions	BookingNotFoundException if ID not found

FR-B04 — List All Bookings

Attribute	Details
Command	list-bookings
Behaviour	Prints all bookings including CANCELLED ones
Edge Case	Prints "No bookings found in the system." if empty

3.3 Recommendation & Referral

FR-REC01 — Recommend Alternative Hotel

Attribute	Details
Trigger	Automatically shown when FR-B01 raises RoomNotAvailableException
Behaviour	Displays list of 3 pre-loaded partner hotels with their available rooms; prompts guest to select
Partner Hotels	KH001 Kunal's Hotel (MG Road), AH001 Arman's Hotel (Mall Road), PH001 Pine View Hotel (Lakkar Bazar)
Guest Choice	[1] / [2] / [3] selects a hotel; [0] declines

FR-REF01 — Record Referral

Attribute	Details
Trigger	Guest selects a partner hotel from recommendation list
Behaviour	Creates ReferralRequest with auto-generated REF-XXX ID and PENDING status; increments partner hotel's customersReferred counter
Side Effect	customersReferred counter is used to calculate commission bills
Referral Date	Automatically set to LocalDate.now()
Exceptions	InvalidInputException for blank name; HotelNotFoundException for null hotel

FR-REF02 — List All Referrals

Attribute	Details
Command	list-referrals
Behaviour	Prints all referrals with ID, customer name, hotel, date, and status

3.4 Commission Billing

FR-C01 — Generate Commission Bill

Attribute	Details
Command	generate-commission <hotelId>
Example	generate-commission KH001
Formula	Total Bill = customersReferred × commissionPerCustomer
Commission Rates	KH001: Rs.1000/customer, AH001: Rs.800/customer, PH001: Rs.500/customer
Default Status	UNPAID
Bill ID	Auto-generated BILL-XXX; kept the same if bill is updated
Update Logic	If bill already exists for hotel, old bill is removed and regenerated with latest count under same ID

Edge Case	Prints informational message and returns null if customersReferred == 0
Exceptions	HotelNotFoundException if hotel is null

FR-C02 — View Commission Bill

Attribute	Details
Command	view-commission <hotelId>
Behaviour	Prints bill ID, hotel details, total customers, rate, total amount, status, and generation date
Exceptions	HotelNotFoundException if no bill exists for given hotel ID

FR-C03 — View All Commission Bills

Attribute	Details
Command	view-all-commissions
Behaviour	Prints all commission bills; "No commission bills found." if none exist

FR-C04 — Mark Commission Paid

Attribute	Details
Command	mark-paid <hotelId>
Behaviour	Sets billStatus to PAID; already-paid bills print a notice without throwing
Exceptions	HotelNotFoundException if no bill exists for hotel

3.5 System / CLI Features

FR-SYS01 — Command Help (Cheat Sheet)

Attribute	Details
Command	admin123 (password)
Behaviour	Prints all 14 available commands with usage and examples to the console; opens AWT cheat-sheet frame on top of other windows
AWT Frame	600×750 px, dark background (30,30,30), gold title, white commands, grey examples; always-on-top; non-resizable

FR-SYS02 — Exit

Attribute	Details
Command	exit
Behaviour	Disposes AWT cheat-sheet frame if open; prints goodbye message; terminates the application

Arun Kumar 09-05-2026

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	All commands follow a consistent verb-noun pattern. Help is accessible via the admin123 password command.
NFR-02	Usability	Guest names with multiple spaces (e.g. "Arun Kumar Sharma") must be accepted and stored correctly.
NFR-03	Reliability	All invalid inputs must throw typed exceptions and print descriptive error messages without crashing.
NFR-04	Reliability	Cancelling an already-cancelled booking or marking an already-paid bill must be handled gracefully.
NFR-05	Correctness	Date overlap detection must prevent double-booking; cancelling a booking must free the dates.
NFR-06	Correctness	Commission formula Total = customersReferred × commissionPerCustomer must always be accurate.
NFR-07	Maintainability	Each service implements a corresponding interface; business logic is isolated from the CLI layer.
NFR-08	Testability	Every service exposes helper methods (getTotalBookings, getLastBookingId, etc.) to support unit testing.
NFR-09	Performance	In-memory ArrayList operations must complete within 1 second for up to 1000 records.
NFR-10	Portability	The application must run on any OS with Java 17+ installed.

5. External Interface Requirements

5.1 User Interface

The primary interface is a text-based CLI. The user types commands at the "hotel> " prompt. An auxiliary AWT window (java.awt.Frame) provides a persistent command reference when the admin123 command is entered.

5.2 Hardware Interfaces

Standard keyboard input and display output. No special hardware is required.

5.3 Software Interfaces

Component	Version	Purpose
Java SE	17+	Runtime, java.time date API, java.awt for cheat-sheet frame
JUnit 5	5.x	Unit testing framework
Maven / Gradle	Any	Build and dependency management

5.4 Communication Interfaces

None. The system is fully standalone with no network communication.

6. System Constraints

Constraint	Description
In-Memory Only	No database; all data is lost on application exit.
Single Session	No multi-user or concurrent access support.
Fixed Partner Hotels	Three partner hotels (KH001, AH001, PH001) are hard-coded in RecommendationService.
Date Format	Only DD-MM-YYYY format is accepted; other formats cause InvalidDateException.
Room Types	Only Single, Double, Deluxe, Suite are accepted by InputValidator.isValidRoomType().
ID Formats	BK-XXX (bookings), REF-XXX (referrals), BILL-XXX (bills); counters start at 001 and are session-scoped.
Language	English only; no internationalisation support.

7. Test Requirements Summary

The following test classes are included in the project under src/test/java/. All tests use JUnit 5 with @BeforeEach setup methods to ensure test isolation.

Test Class	Tests	Key Scenarios Covered
RoomServiceTest	30	addRoom (success + 6 validation failures), deleteRoom, getRoom, getAvailableRooms, isRoomAvailable, display methods
BookingServiceTest	35	bookRoom (happy path, multi-word name, non-overlapping dates, 9 validation failures), cancelBooking, getBooking, display methods
CommissionServiceTest	28	generateBill (formula, update, null hotel, zero referrals), getBill, markBillAsPaid, formula verification (10×1000, 5×800)
ReferralServiceTest	24	addReferral (success, counter increment, ID format, validation failures), getReferralsByHotel, confirmReferral
RecommendationServiceTest	20	Preloaded hotel count, hotel room counts, commission rates, getHotel (case-sensitive), hasAvailableHotels
InputValidatorTest	55	All 11 validator methods: isValidInput, isPositiveNumber, isValidRoomNumber, isValidRoomType, isValidDate, isValidDateRange, isValidGuestName, isValidBookingId, isValidReferralId, isValidPrice, isValidHotelId